

Efficient Ensemble for Fine-tuning Language Models on Multiple Datasets

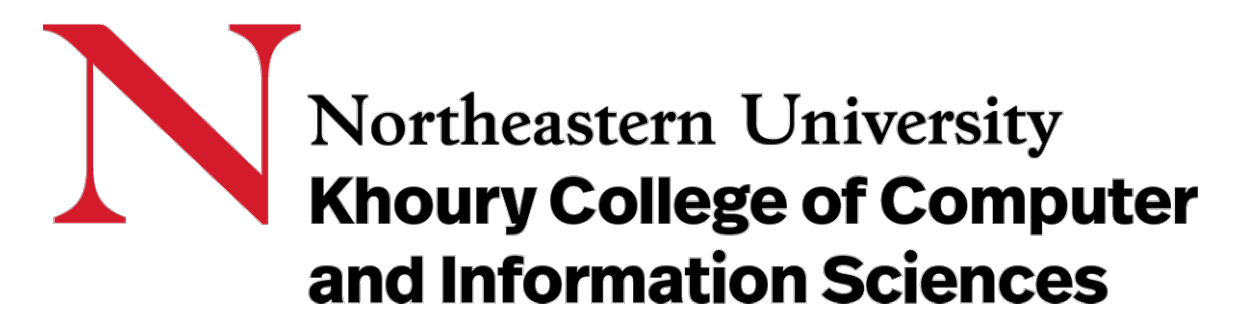


Dongyue Li[†], Ziniu Zhang[†], Lu Wang[‡], Hongyang R. Zhang[†]

[†]Northeastern University, Boston

[‡]University of Michigan, Ann Arbor

[†]{li.dongyu, zhang.zini, ho.zhang}@northeastern.edu [‡]wangluxy@umich.edu



BACKGROUND AND PROBLEM STATEMENT

Parameter-efficient fine-tuning has emerged as an efficient approach to adapting LLMs to a downstream application, such as LoRA (Hu et al., 2021) and Adapter Tuning (Houlsby et al., 2019).

Fine-tuning LMs on a mix of many datasets appears in many applications.

- Directly applying a base fine-tuning method on all tasks, which can lead to negative interference across tasks
- Pretraining a shared adapter and fine-tuning it on each task, which increases the computational cost by two

Problem statement: Given n datasets and a base adaptation method, we aim to design an ensemble of adapters that applies weighted averaging to their outputs, with minimal computation and memory overhead to the base method.

- First, we partition n datasets into m groups based on estimating task affinities.
- Second, we design an ensemble of adapters by fine-tuning one adapter per group, denoted as $\{\theta_1, \theta_2, \dots, \theta_m\}$, and combining the outputs with weights $\{w_1, w_2, \dots, w_m\}$.

RELATED WORKS

Influence functions evaluate how slight changes to individual training example weights affect model predictions (Koh and Liang, 2017).

Data modeling fits surrogate models via random sampling to approximate the loss of the model trained on a training sample subset (Park et al., 2023; Ilyas et al., 2022).

Task affinity grouping computes a task similarity score by computing gradient similarity between tasks (Fifty et al., 2021).

OUR APPROACH

Task affinity grouping for LM fine-tuning: First, we partition the n given datasets into m disjoint subsets based on the task affinities.

- Let S be a subset of datasets. Let $f_i(S)$ be the fine-tuning performance on dataset i using all datasets in S .
- The task affinity score of task j to i is: $T_{i,j} = \frac{1}{n_{i,j}} \sum_{S_k} f_i(S_k)$, where $\{S_k\}$ are subsets sampled from $\{1, \dots, n\}$ and $n_{i,j}$ is the number of random subsets that contain both i, j

Computing $T_{i,j}$ requires fine-tuning the LM on multiple subset combinations. How can we design an efficient algorithm to quickly estimate $f_i(S)$?

A gradient-based estimation algorithm for LM fine-tuning losses: We propose an estimation algorithm for LM fine-tuning losses on subsets of tasks without performing any fine-tuning.

- We observe that the distance of the fine-tuned adapter to the pretrained model is less than **0.2%** relative to all pretrained model weights on average.
- When the fine-tuned adapter parameter X is close to the initialization θ^* , the model output $h_X(s, y)$ can be approximated by a first-order Taylor's expansion centered at the meta-initialization θ^* .

$$h_X(s, y) \approx h_{\theta^*}(s, y) + [\nabla_X h_{\theta^*}(s, y)]^\top (X - \theta^*) + \epsilon_s$$

The Relative RSS is less than **3%** on average.

Thus, we apply the gradient-based approximation to the log-loss of the model:

$$\tilde{\ell}(X) = \log(1 + \exp(b - y \cdot g^\top (X - \theta^*)))$$

The estimation corresponds to solving logistic regression using the gradient g_i as the feature vector and the task label y as the response label.

Solving each logistic regression takes within **2** seconds per instance on CPUs with projected gradients!

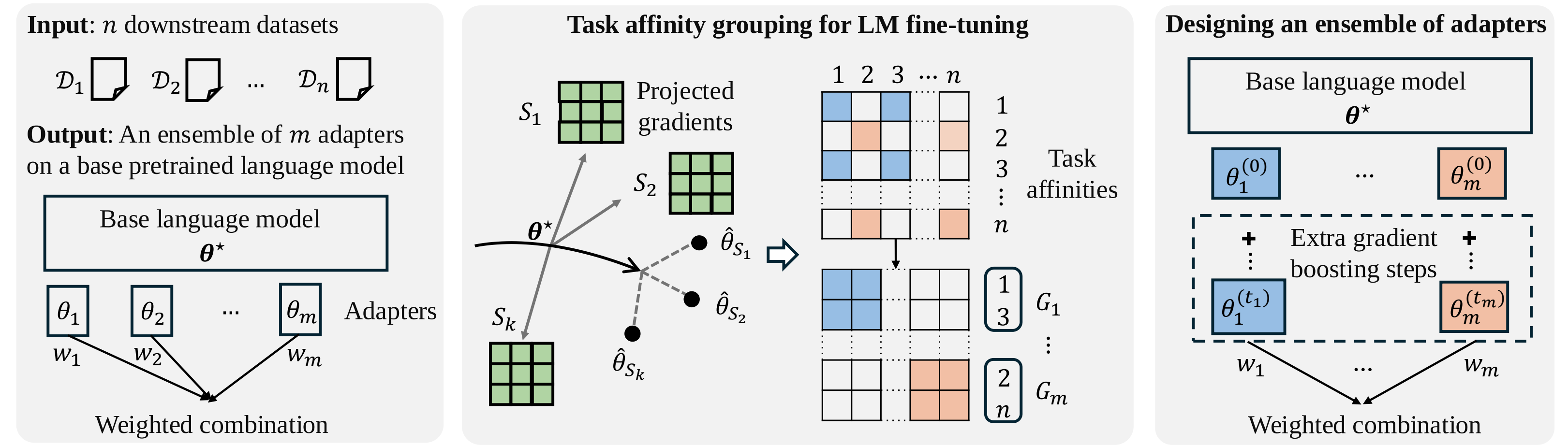
Lastly, we compute task affinity scores for every pair of tasks. Given a $n \times n$ task affinity matrix, we apply a clustering algorithm to partition n datasets into m groups.

Designing an ensemble of adapters:

- We fine-tune one adapter on tasks in each group, yielding m adapters.
- We apply b additional gradient boosting steps to reduce the loss of groups with high training losses. In our experiments, the first boosting step leads to **??0.4%** age test accuracy improvement for tasks in the group.

The final ensemble contains a total of $M = m + b$ adapters.

SUMMARY OF OUR APPROACH



We present **EnsembleLoRA**, which constructs an ensemble of low-rank adapters for fine-tuning an LM on multiple datasets.

- Compared to the base fine-tuning method, the additional cost of our approach is only evaluating the gradients once for all the samples.
- Compared to pretraining followed by fine-tuning, our method saves the computational cost by half and GPU memory by $\frac{n}{M}$.

EXPERIMENTS

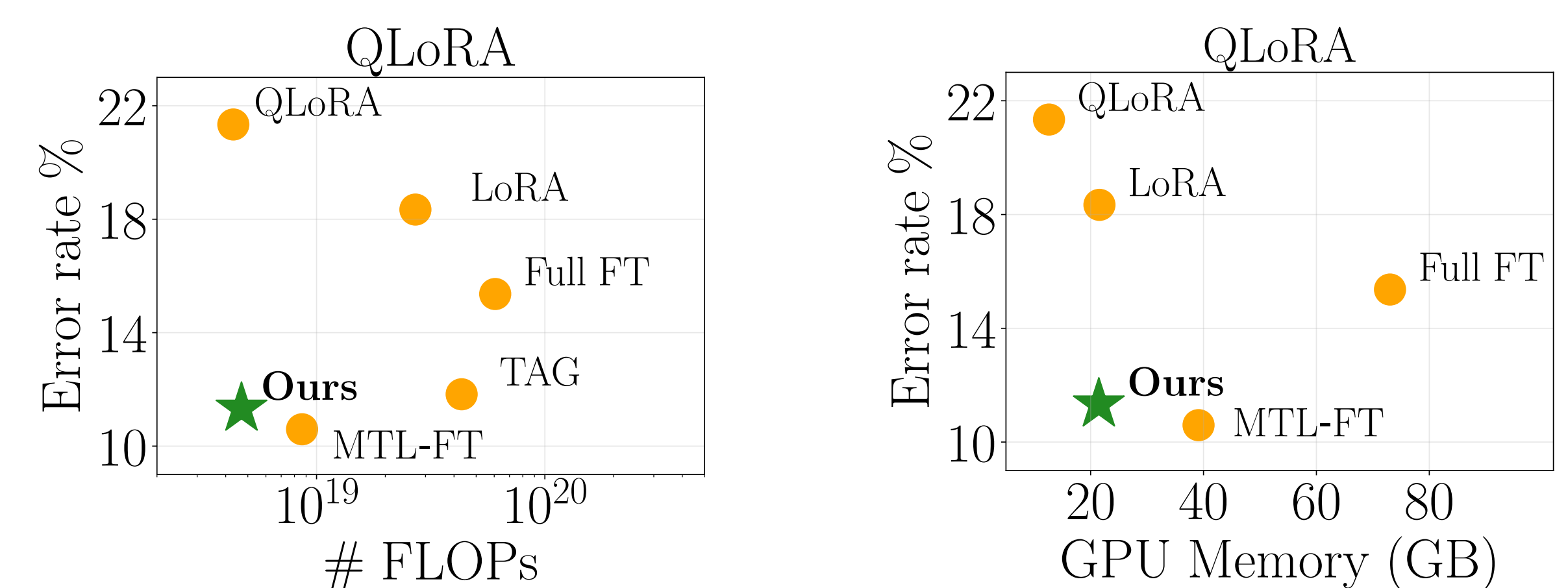
Approximating model fine-tuning losses: We evaluate the error between estimated and true fine-tuning performances on 50 random task subsets. We vary the projected dimension from 200 to 800.

- Our approach achieves a relative error within **5%** while using **105×** less computation than actual fine-tuning.

d	Llama-3-1B	Llama-3-3B	Llama-3-8B	Speedup
200	8.2%	8.1%	7.0%	105×
400	4.7%	4.8%	4.3%	105×
800	4.6%	4.4%	4.2%	105×

Trade-off of the test accuracy and computational/memory cost:

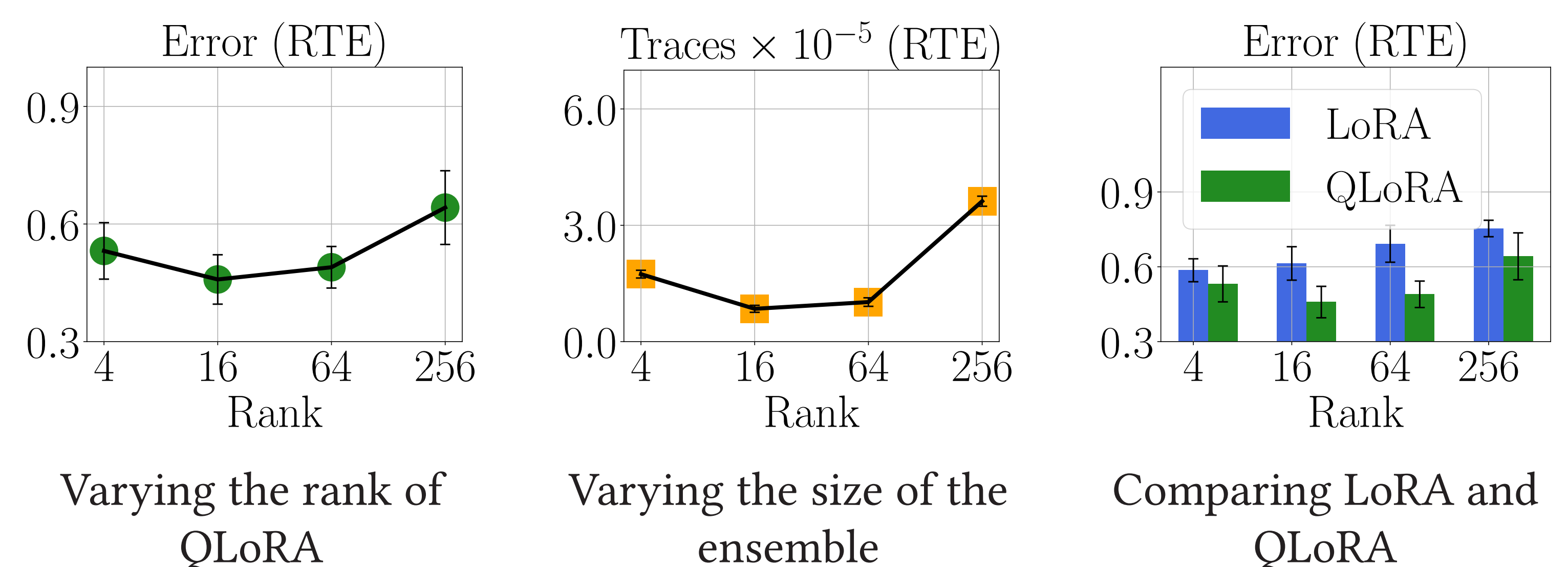
- Compared to QLoRA, we improve accuracy by **10%** with only **8%** additional computation and **9** GB more memory.
- Compared to pretraining followed by fine-tuning and task grouping, we achieve comparable performance while reducing computation and memory by **45%**
- Compared to full model fine-tuning, we improve accuracy by **4%** with **92%** less computation and **52** GB less memory.



Trade-off of error rates with computational and memory cost

Generalization of low-rank adaptation: We evaluate the empirical **generalization errors** and **sharpness** by the trace of the Hessian.

- Relatively small adapters often achieve the best generalization performance.
- Ensembles of adapters yield lower generalization errors than individual adapters.
- Adapters trained on quantized models exhibit smaller generalization errors compared to those trained on full-precision models.



Varying the rank of QLoRA

Varying the size of the ensemble

Comparing LoRA and QLoRA

Code: <https://github.com/VirtuosoResearch/EnsembleLoRA>

Paper: <https://arxiv.org/pdf/2505.21930>

Financial support from the NSF award IIS-2412008 and JP Morgan Chase.